endava

Advanced GitHub Copilot for real codebases

Potential problems faced when using GenAI

Vague prompts

The request is underspecified, so the output looks plausible but but misses business intent. intent.

Missing context

Copilot only sees a fragment of the repo, so it guesses the surrounding architecture.

Weak validation

Generated code is accepted before tests, linting, and human review catch the hidden issues.

No shared guidance guidance

Every developer uses different habits because because repository instructions and policies policies are missing.

Ask yourself

1/ Where does Copilot fail most often in our workflow today?

2/ Which of these four causes feels most expensive for us?

Agenda

01

Prompting

How to write prompts that behave like engineering specifications.

02

Context

How Chat reasons across files, symbols, open tabs, and repository indexes.

03

Patterns

How to guide generation generation with existing existing architecture and and API usage examples. examples.

04

Tests + Refactors

How to expand coverage coverage and change concepts safely across across modules.

05

Workflow + Governance

How IDE habits, instructions, policies, and and model controls keep keep output consistent. consistent.

Shared Repository

Demo repo

A TypeScript commerce API with four areas: orders, pricing, users, and billing. Every demo reuses the same repo so everyone can see how prompts, context, tests, and refactors connect.

```
src/
  features/
    orders/
      create-order.controller.ts
      create-order.service.ts
      order.repository.ts
    users/
      create-user.controller.ts
      create-user.service.ts
    pricing/
      price-calculator.ts
  shared/
    auth/
    telemetry/
    validation/
```

The operating model

Copilot quality is not magic. It is the output of four controllable inputs.

Prompt

State the goal, constraints, expected output, and validation rules.

quality = prompt × context × patterns × verification

Context

Expose the right files, symbols, examples, and repository history.

Patterns

Give Copilot architecture to imitate instead of asking it to invent one.

Verification

Use tests, linting, code review, and domain judgment before accepting output.

01

Authoring effective prompts prompts

What strong prompts have in common

GitHub recommends starting general, then getting specific. For code, that means acting like you are writing acceptance criteria.

Task

What should Copilot do? Generate, explain, refactor, compare, or review?

Constraints

Language, framework, layering, dependencies, style, performance, security.

Context

Relevant files, patterns to follow, domain assumptions, and non-obvious rules.

Output

What artifact do you want back: code, plan, tests, summary, or migration steps?

Validation

How should the answer be checked: tests, lint, error cases, or trade-off analysis?

Rule of thumb

If an engineer could not implement the task from your request alone, Copilot will also be forced to guess.

Weak prompt versus strong prompt

Same task. Radically different output quality.

Weak

```
Create an endpoint to create orders.
```

Likely result

Boilerplate with guessed conventions and weak test coverage.

Strong

```
Create a POST /orders endpoint in TypeScript.

Constraints:
- follow the existing controller/service/repository pattern
- reuse the validation and telemetry approach from CreateUser
- return the standard 201 response envelope
- do not add dependencies

Also generate Jest tests for:
- happy path
- invalid payload
- duplicate order ID
- repository failure
```

Likely result

A repo-shaped answer that aligns to existing patterns and validation expectations.

Reusable prompt recipes

Generate

Create X following Y pattern, with Z constraints, and include include tests for A/B/C. A/B/C.

Transform

Refactor this code to achieve X while preserving behavior and and existing interfaces. interfaces.

Explain

Trace how this feature works across files and identify the extension points.

Compare

Show two approaches, trade-offs, and the safer safer option for this codebase.

Interactive demo 1: prompt clinic

```
Task for the room

"Add a create-order endpoint to the sample repo."

Before you run Copilot:
- what details are missing?
- which existing files should shape the answer?
- which tests must be requested explicitly?
```

Ask yourself

1/ What must be specified so Copilot does not invent a new architecture?

2/ What failure cases would you insist on in the first generated test set?

02

Context across large codebases

Choose the right Copilot surface for the job

Inline suggestions, chat, and agent-style flows are related, but they are not interchangeable.

Inline

Best for small completions, repetitive boilerplate, and comment-to-code generation while you stay in flow.

Chat

Best for natural-language reasoning, reasoning, larger code generation, generation, explanation, and iterative improvement.

Agentic workflows

Best for repo-wide or multi-step tasks that need search, planning, writing, and verification together.

Multi-file and project-level tactics that actually help

Most context failures are workflow failures, not model failures.

Open the neighbors

Keep the relevant controller, service, repository, and shared utilities open. Close unrelated tabs.

Reference explicitly

Use file references or drag the exact file into chat when the answer must anchor to a specific module.

Reset stale context

If the conversation history is now noise, start a new chat instead of fighting the old one.

Check references

After each answer, inspect which files Copilot used before trusting the result.

```
1 @workspace Trace how authorization is
enforced for write operations.
```

Small but powerful move

If you care about one answer being repo-accurate, spend 20 seconds curating context curating context before you ask.

Interactive demo 2: contextual reasoning reasoning over the repo

```
@workspace Trace how authorization is enforced for write operations.

Map the flow from the HTTP entry point to the service layer.
Identify the best extension point for tenant-aware authorization.
List the files I should review first.
```

Ask yourself

1/ What would make this answer trustworthy enough to act on?

2/ Which files would you expect to appear in References if the answer is answer is good?

3/ Would you ask for a code change yet, or first ask for a trace and a plan? a plan?

03

Pattern-guided generation generation

Copilot imitates patterns more reliably than it invents architecture

Best inputs for pattern-guided generation generation

- an existing feature in the same repo
- the preferred layering and naming scheme
- one correct API usage example
- the testing style you want repeated
- the response envelope and error mapping
- a note about what must not change

Controller
HTTP mapping
validation

→

Service
domain logic
telemetry

→

Repo
persistence

Prompt language that works

“Generate create-order following the same controller/service/repository pattern as create-user. Reuse the validation, telemetry, naming, and response envelope conventions.”

Interactive demo 3: pattern reuse

The best prompt is often “follow this exact repo pattern,” not “be creative.”

```
1 Using src/features/users/create-user as the
reference implementation,
2 generate src/features/orders/create-order.
3
4 Follow the same layering, naming conventions,
validation approach,
5 mapper style, telemetry hooks, and tests.
6
7 Call out any assumptions before writing code.
```

Ask yourself

1/ Which exact file should be the exemplar — controller, service, or full full feature slice?

2/ What outdated patterns might Copilot accidentally copy forward? forward?

3/ What assumptions would you want Copilot to surface before it writes writes anything?

04

Tests and semantic refactors

Automated test expansion is high-ROI Copilot work

GitHub and VS Code both highlight testing as a strong fit because the task is repetitive, pattern-rich, and easy to validate.

Where Copilot helps most

- unit tests
- integration tests
- end-to-end scaffolding
- framework setup
- fixing failing tests
- making style consistent across suites

Prompt for edge cases explicitly

Ask for boundary values, null/empty inputs, invalid invalid states, error paths, permission failures, failures, retry behavior, and dependency failures. failures. The model is good at enumeration only only when you name the categories.

```
1 Generate additional tests for
PriceCalculator.
2
3 Include:
4 - boundary values
5 - null/empty inputs
6 - invalid currency
7 - rounding edge cases
8 - dependency failures
```

Edge-case checklist

Boundary values

min/max, zero, overflow, rounding

Bad inputs

null, empty, malformed, wrong enum

State errors

duplicates, missing records, stale version

Dependency failures

repo throws, timeout, bad gateway, bad bad parse

Security / auth

unauthorized, forbidden, tenant mismatch

Behavioral quirks

idempotency, concurrency, retries, ordering

Semantic refactoring across modules needs a safer workflow

Use Copilot for impact analysis and draft changes — but only with clear guardrails.

1 Map impact

Ask Copilot to list files, contracts, tests, and likely breakpoints.

2 Choose scope

Rename, extract, migrate types, or standardize behavior — one change at a time.

3 Change incrementally

Work in small diffs instead of asking for one giant repo rewrite.

4 Regenerate tests

Ask for missing test updates and new coverage around changed behaviors.

5 Run verification

Execute tests, linting, and code scanning before accepting anything.

6 Review semantics

Check that business meaning stayed correct, not just compile success.

Interactive demo 4: tests first, refactor second

```
Generate additional tests for PriceCalculator.

Then propose a safe refactor plan to replace number-based
money handling with Decimal across the impacted modules.
List affected files first.
```

Ask yourself

1/ Should Copilot refactor immediately, or first enumerate impact and test gaps?

2/ Which modules will probably break even if the compiler stays green?

3/ What is the smallest first slice of this refactor you would accept?

05

Workflow and governance governance

Customize Copilot in the repository, not only in the prompt

Persistent guidance reduces repetition and makes team behavior more consistent.

```
.github/
  copilot-instructions.md
  instructions/
    tests.instructions.md
    frontend.instructions.md
  prompts/
    create-endpoint.prompt.md
AGENTS.md
```

Repo-wide

Use .github/copilot-instructions.md for shared shared architecture, libraries, libraries, naming, and safety safety rules.

Path-specific

Use *.instructions.md files when test code, frontend code, or infra code needs different guidance.

Reusable prompts

Use .github/prompts/*.prompt.mt.md for repeatable tasks like like endpoint creation, review, or migration.

Important caveat

In VS Code, custom instructions influence chat requests, but not but not raw inline suggestions as you type.

Good habit

Keep instructions short, concrete, and example-driven. Use file-specific instructions only where conventions truly diverge.

Repository settings and org policies complete the picture

Prompt quality improves the answer. Governance controls where and how Copilot can operate.

Repository / organization controls

- enable or disable Copilot features
- control access to additional AI models
- opt into previews carefully
- manage content exclusion for sensitive paths
- audit changes to exclusion settings
- align partner/third-party agents to repo access rules

Content exclusion

Admins can tell Copilot to ignore selected repository content.

Model controls

Organizations can restrict which Copilot models are available beyond the basic set.

Common anti-patterns

These are the mistakes that make Copilot look smarter or worse than it really is.

Treating prompts as casual requests

If the instruction is fuzzy, the result is forced to guess.

Expecting repo accuracy without curation

Open files, references, and instructions still matter.

Asking for giant refactors in one one shot

Smaller slices are easier to verify and safer to review.

Accepting generated tests without reading them

Shallow assertions and false confidence are common.

Copying stale patterns from the codebase

Copilot will faithfully preserve technical debt if it is the nearest exemplar.

Skipping verification because the code compiles

Readable, maintainable, secure, and semantically correct are different questions.

Five operating rules for production-grade Copilot use

Rule 1

Prompt like a spec, not not a suggestion.

Rule 2

Expose context intentionally before asking.

Rule 3

Seed good patterns before generating new code.

Rule 4

Ask for tests and edge edge cases early.

Rule 5

Verify everything important with tools and judgment.

endava

Thank you

Q&A · workshop discussion · demo follow-ups